DEEP LEARNING 2026 · UNIVERSIDAD DEL VALLE DE GUATEMALA · PROYECTO 3

Text-to-SQL en español con LoRA

Fine-tuning de Qwen2.5-Coder-1.5B-Instruct para generar consultas MySQL 8.4 a partir de un esquema y una pregunta

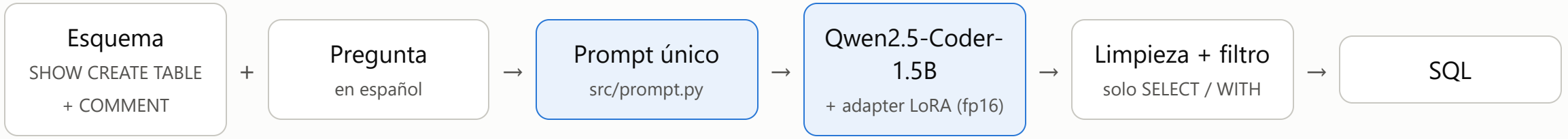
Carlos Alburez
Datos · validación propia · evaluador

Erick Guerra
Entrenamiento LoRA · inferencia · experimentos

0.80

EX ponderado del modelo entregado (LoRA r8 v3) en nuestra validación propia

El problema y el contrato de entrega



Restricciones del curso

- `generar_sql(esquema, pregunta, ejecutar=None) -> str`
- ≤ 30 s por pregunta · ≤ 5 min de carga · Colab T4
- Determinista; nunca una consulta que modifique la base
- Métrica: exactitud por ejecución ponderada (fácil 1, media 2, difícil 3)

Cómo lo cumple `solucion.py`

- Greedy (`do_sample=False`), semilla 2026: **verificado idéntico en dos corridas**
- Carga en 7–13 s; 2.1 s promedio y 20.2 s máximo por pregunta (RTX 4060)
- `es_solo_lectura()`: 40 pruebas y 0 falsos rechazos sobre 2,469 consultas reales
- Con `ejecutar`: hasta 3 intentos de autocorrección

Datos: Spider en español + 12 esquemas propios

2,429

pares de Spider + MultiSpider ES que ejecutan en MySQL 8.4.10 (de 7,101)

460

pares propios sobre 12 esquemas, generados con LLM; 0 descartes

3,349

ejemplos en el dataset final (v3)

Spider: conversión SQLite → MySQL con `sqlglot` y **filtro por ejecución real** con `ONLY_FULL_GROUP_BY` (65 bases). Propios: nombres en español, inglés y abreviados, con COMMENT; 205 difíciles con fechas, `WITH` y ventanas. Cada par se ejecutó sobre la base poblada con datos sintéticos.

4,672 descartes de Spider y su motivo

Motivo	Pares
Tabla inexistente (FK complejas al cargar)	3,750
Columna inexistente	854
Violación de ONLY_FULL_GROUP_BY	42
Otros errores de ejecución	24
Función no soportada	2

Detalle y ejemplos en `datos/descartes/resumen_descartes.md`

Validación propia y dos correcciones al evaluador

Diseño

- 2 esquemas escritos por nosotros: `sistema_academico` y `clinica_salud`, con COMMENT
- 40 preguntas: 15 fáciles, 15 medias, 10 difíciles (WITH, ventanas, fechas)
- Partición por esquema: **0 % de fuga** (`src/chequeo_fuga.py`)
- EX por ejecución: multiconjunto, 2 decimales, columnas extra permitidas

Lo que encontramos al evaluar el baseline

- **Solo se comparaban 10 filas.** 9 de 40 preguntas devuelven más; sin ORDER BY eso da falsos aciertos y falsos errores → el evaluador ahora compara el resultado completo.
- **Las preguntas no decían qué columnas devolver**, a diferencia de la evaluación oculta. 18 de 23 errores del baseline eran eso. Reescribimos el texto (no las referencias): baseline **0.387** → **0.747**.

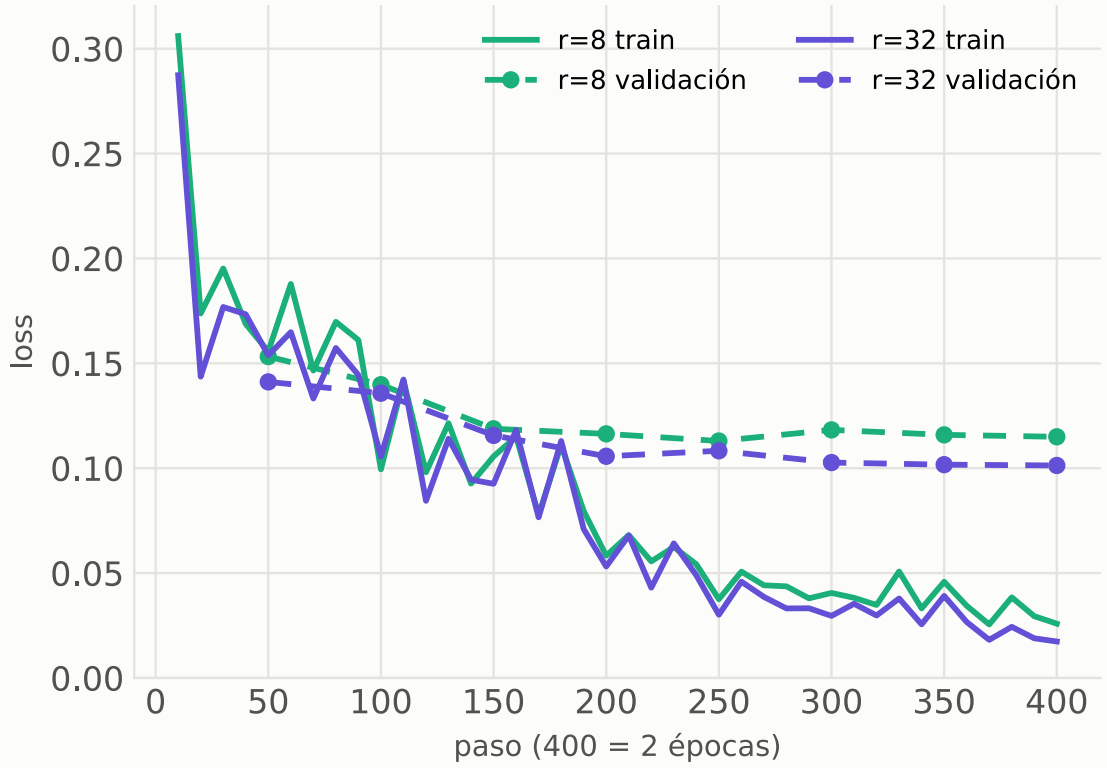
Experimento LoRA: un solo factor, el rango r

	Config 1	Config 2
r / alpha	8 / 16	32 / 64
target_modules	q,k,v,o,gate,up,down	igual
learning rate · épocas	2e-4 · 2	igual
batch efectivo	16 (2 × 8)	igual
loss	solo sobre el SQL	igual
hardware	1× Tesla T4 (Kaggle)	igual

Hipótesis (escritas antes de entrenar)

1. **H1** r=32 baja más la loss de entrenamiento.
2. **H2** Con 2.4k pares la diferencia de EX será ≤ 5 pts; si r=32 sobreajusta, r=8 generaliza igual o mejor.
3. **H3** Si r=32 gana, será en preguntas difíciles.
4. **H4** r=32 tarda solo +10–20 %.

Entrenamiento: tiempo, loss y curvas (dataset v3)

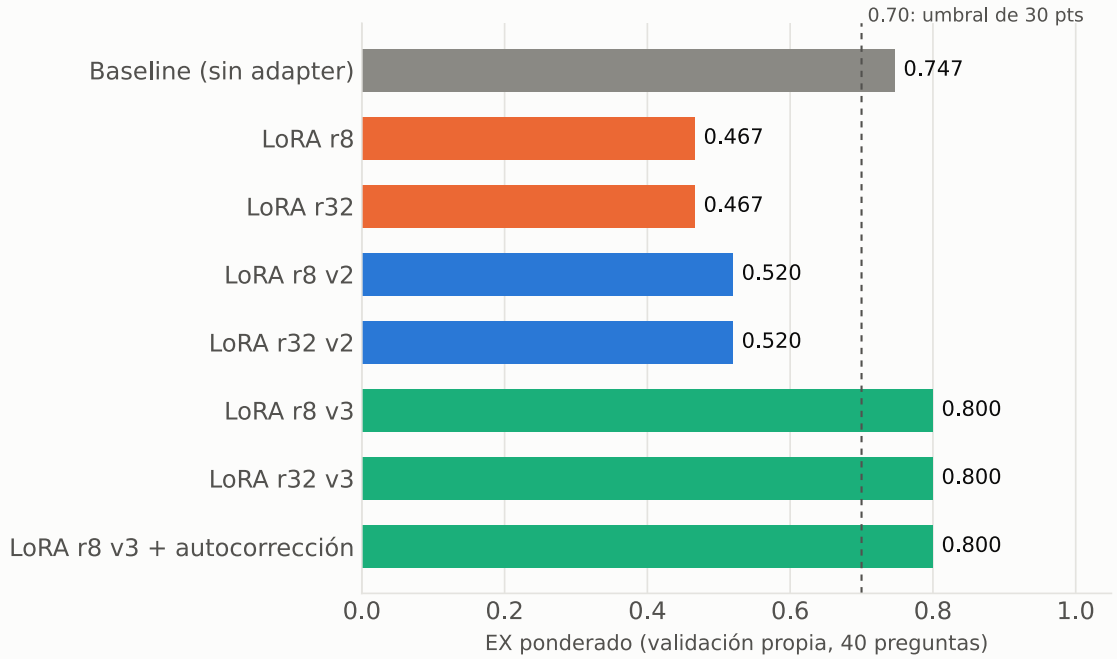


Dataset · config	Ejemplos	Tiempo T4	Loss train	Loss val mín / final	Adapter
v1 · r = 8	2,359	58.3 min	0.045	0.111 / 0.143	18.5 MB
v1 · r = 32	2,359	58.8 min	0.027	0.123 / 0.153	73.9 MB
v2 · r = 8	2,357	59.7 min	0.037	0.100 / 0.108	18.5 MB
v2 · r = 32	2,357	60.3 min	0.024	0.107 / 0.110	73.9 MB
v3 · r = 8	**3,191**	**95.3 min**	**0.026**	**0.113 / 0.115**	**18.5 MB**
v3 · r = 32	3,191	96.1 min	0.017	0.101 / 0.101	73.9 MB

Parámetros entrenables: r = 8 → 9.2 M (0.59 %), r = 32 → 36.9 M (2.34 %). Adapters en fp16.

La loss de validación casi no baja en la época 2, pero **el EX de la época 2 siempre es igual o mejor** que el de la época 1: la loss no basta para elegir el adapter.

Resultados: EX en la validación propia



Versión	EX	Fácil / media / difícil	Época 1	vs baseline
Baseline (sin adapter)	**0.747**	1.00 / 0.87 / 0.50	—	—
LoRA r8	**0.467**	1.00 / 0.47 / 0.20	—	-0.280
LoRA r32	**0.467**	1.00 / 0.47 / 0.20	—	-0.280
LoRA r8 v2	**0.520**	1.00 / 0.60 / 0.20	0.480	-0.227
LoRA r32 v2	**0.520**	1.00 / 0.60 / 0.20	0.467	-0.227
LoRA r8 v3	**0.800**	**1.00 / 0.80 / 0.70**	**0.707**	**+0.053**
LoRA r32 v3	**0.800**	1.00 / 0.80 / 0.70	0.720	+0.053

Gris: modelo base · naranja: dataset v1 · azul: v2 (indica columnas) · verde: v3 (alias legibles + pares propios MySQL).

Hipótesis contrastadas y evolución del dataset

	Resultado	Evidencia (v1, v2, v3)
H1	✓ confirmada	loss train r=32 vs r=8: 0.027 vs 0.045, 0.024 vs 0.037, 0.017 vs 0.026
H2	✓ confirmada	ΔEX (r=32 − r=8) por dataset: 0, 0, 0
H3	— no aplica	r=32 no gana en ningún dataset
H4	✗ refutada (a favor)	tiempo extra de r=32: +1%, +1%, +1%
H5	✓ confirmada	v3: 0.800 vs baseline 0.747

H1–H4 sobre el rango, escritas antes del primer entrenamiento. H5 (dataset v3) escrita antes de entrenar v3: igualar o superar el baseline.

v1 → v2: indicar columnas

El LoRA v1 ignoraba "Devuelve las columnas: …" (Spider nunca lo dice): 10–11 de 16 errores eran de columnas. v2 agrega la indicación y alias a las expresiones (761 consultas re-ejecutadas). EX 0.467 → 0.520, aún bajo el baseline.

v2 → v3: quitar el estilo de Spider

v2 rompía 8 preguntas que el base acertaba: alias `T1` / `T2` que confunden tablas, sin funciones de fecha (`2026 - fecha_nac` en vez de `TIMESTAMPDIFF`), JOIN y GROUP BY pobres. v3: alias legibles (784 consultas) + 460 pares MySQL propios.

Experimento de inferencia

Experimento: autocorrección con `ejecutar`

Si MySQL devuelve error, se le muestra al modelo la consulta y el mensaje, y se le pide corregirla (máx. 3 llamadas).

Hipótesis: reduce los errores de ejecución; no ayuda cuando la consulta corre pero el resultado es incorrecto (no hay señal de error).

Versión	EX	No ejecutan	Tiempo medio
LoRA r8 v3	0.800	3	2.9 s
LoRA r8 v3 + autocorrección	0.800	3	3.7 s

Resultado

- Llamadas a `ejecutar` por pregunta: 1: 37, 3: 3
- Preguntas arregladas: **0**
- Los errores restantes son mayoritariamente de lógica o columnas, que `ejecutar` no detecta; un modelo de 1.5B tiende a repetir la misma consulta ante el mismo error.

Análisis de errores por categoría

Versión	Sintaxis	Tabla inexist.	Columna	JOIN	Agregación	Filtro	Resultado distinto	Total
Baseline (sin adapter)	·	·	3	·	·	·	4	7
LoRA r8	·	·	10	1	·	·	5	16
LoRA r8 v2	·	·	6	·	·	·	8	14
LoRA r8 v3	·	·	2	·	1	·	3	6

"Columna" incluye consultas que ejecutan pero no devuelven las columnas pedidas o les ponen otro nombre.

Ejemplos (LoRA r8 v3)

val_11 · media · resultado_distinto — ¿Cuál es el promedio de calificaciones de los estudiantes en cada curso? Devuelve las columnas: nombre, promedio_nota.

```
pred: SELECT c.nombre, ROUND(AVG(i.calificacion), 2) AS promedio_nota FROM cursos c JOIN inscripciones i ON i.curso_id = c.id GROUP BY c.id, c.nombre
```

```
ref:  SELECT c.nombre, ROUND(AVG(i.calificacion), 2) AS promedio_nota FROM cursos c JOIN inscripciones i ON c.id = i.curso_id WHERE i.calificacion IS NOT NULL GROUP BY c.id, c.nombre
```

val_15 · media · columna_inexistente — ¿Cuál es el total de créditos inscritos por cada estudiante en el semestre 2024-1? Devuelve las columnas: nombre, total_creditos.

```
pred: SELECT e.nombre, SUM(i.creditos) AS total_creditos FROM estudiantes e JOIN inscripciones i ON i.estudiante_id = e.id WHERE i.semestre = '2024-1' GROUP BY e.id, e.nombre
```

```
ref:  SELECT e.nombre, SUM(c.creditos) AS total_creditos FROM estudiantes e JOIN inscripciones i ON e.id = i.estudiante_id JOIN cursos c ON i.curso_id = c.id WHERE c.semestre = '2024-1' GROUP BY e.id, e.nombre
```

Lo que aprendimos

- **Los datos importan más que la capacidad del adapter.** Con cada dataset r=8 y r=32 empatan en EX; cambiar los datos movió el EX de 0.47 a 0.80.
- **El fine-tuning puede empeorar a un modelo instruido.** Con Spider tal cual, el LoRA desaprendió cosas que el base sabía (seguir la indicación de columnas, funciones de fecha). Solo superó al base cuando los datos se parecieron a la tarea real.
- **Comparar pregunta por pregunta contra el baseline** dijo qué arreglar en los datos; el EX global solo decía que algo iba mal.
- **Validar el evaluador es parte del experimento.** Comparar solo 10 filas y preguntas sin columnas cambiaban la conclusión.
- **Ingeniería:** subir cada adapter apenas termina (perdimos 1 h en Colab) y fijar versiones (torchao, transformers 5) protege la entrega en el Colab del docente.

Predicción justificada del EX en la evaluación oculta

≈ 0.70

EX predicho con adapter

≈ +0.04

mejora predicha sobre el modelo sin adapter

Validación propia: EX 0.800, IC 95 % por bootstrap sobre las 40 preguntas: [0.65, 0.93]. Mejora +0.053, IC 95 % pareado [-0.11, +0.22].

Justificación

- **Punto de partida:** validación propia 0.800 con adapter vs 0.747 sin él. El adapter gana 3 preguntas (val_19 dificil, val_37 dificil, val_39 dificil) y pierde 2 (val_34 media, val_38 dificil): la mejora está en las difíciles, que pesan 3.
- **Sesgo optimista:** elegimos el dataset (v1 → v2 → v3) mirando esta misma validación, así que su EX sobreestima. Además son 40 preguntas: IC 95 % de la mejora [-0.11, +0.22], que incluye 0.
- **Esquemas más difíciles:** la oculta tiene 5–10 tablas y nombres abreviados o en inglés; los nuestros, 5–6 tablas en español. Esperamos unos 10 puntos menos en ambas versiones; v3 incluye esquemas en inglés y abreviados para amortiguarlo.
- **Lo que se mantiene:** el estilo de la oculta (indica columnas, fechas, `WITH`, ventanas) es justo lo que agregó v3, así que esperamos que la ventaja sobre el modelo base se conserve, aunque más pequeña.